

Lecture on Alpha-Beta Pruning

Course: Artificial Intelligence

Instructor: Syed Hamed Raza

Overview of the Lecture:

This lecture will focus on **Alpha-Beta Pruning**, an optimization technique for the **Minimax Algorithm**. Alpha-Beta Pruning helps improve the efficiency of the Minimax algorithm by pruning (discarding) branches in the game tree that cannot influence the final decision. By eliminating unnecessary branches, Alpha-Beta Pruning reduces the number of nodes evaluated, allowing the search to be completed much faster. By the end of this lecture, you will understand how Alpha-Beta Pruning works, its applications, and its significance in game-playing AI.

Agenda:

1. **Introduction to Alpha-Beta Pruning**
 2. **How Alpha-Beta Pruning Works**
 3. **Alpha-Beta Pruning Pseudocode**
 4. **Optimality of Alpha-Beta Pruning**
 5. **Example of Alpha-Beta Pruning in Action**
 6. **Classroom Discussion and Problem Solving**
-

1. Introduction to Alpha-Beta Pruning

What is Alpha-Beta Pruning?

- **Alpha-Beta Pruning** is an optimization technique for the **Minimax Algorithm** that eliminates branches of the search tree that cannot possibly affect the final decision. It is used to improve the efficiency of the Minimax algorithm by reducing the number of nodes evaluated.
- **Alpha-Beta Pruning** does not change the outcome of the Minimax algorithm; it just makes the algorithm faster by pruning away parts of the tree that do not need to be explored.

Why Use Alpha-Beta Pruning?

- **Minimax** searches through a large number of nodes in the game tree. For example, in a game like Chess or Tic-Tac-Toe, the tree can expand exponentially, making it inefficient to search through all nodes.
 - By pruning unnecessary branches, Alpha-Beta Pruning can cut the number of nodes to be explored significantly, improving the runtime without affecting the outcome of the algorithm.
-

2. How Alpha-Beta Pruning Works

The key idea behind **Alpha-Beta Pruning** is to maintain two values, **Alpha** and **Beta**, during the search process:

- **Alpha** represents the best value found so far along the path of the **Maximizing player** (MAX player).
- **Beta** represents the best value found so far along the path of the **Minimizing player** (MIN player).

Alpha:

- The maximum value that the maximizing player is assured of. As the search progresses, **Alpha** is updated to be the maximum of the current **Alpha** and the value of the current node.

Beta:

- The minimum value that the minimizing player is assured of. As the search progresses, **Beta** is updated to be the minimum of the current **Beta** and the value of the current node.

Pruning Conditions:

- If **Alpha** $\geq$ **Beta** at any point in the search, then the current branch can be pruned, because there is no need to explore this branch further. This means that the outcome of this branch will not affect the decision of the player making the current move.

3. Alpha-Beta Pruning Pseudocode

The **Alpha-Beta Pruning** algorithm works as an enhancement to the basic **Minimax Algorithm** by keeping track of **Alpha** and **Beta** values and pruning branches when necessary. The pseudocode for the **Alpha-Beta Pruning** algorithm is as follows:

```
function AlphaBeta(node, depth, alpha, beta, maximizingPlayer):
    if depth == 0 or node is a terminal node:
        return evaluate(node)

    if maximizingPlayer:
        maxEval = -∞
        for each child of node:
            eval = AlphaBeta(child, depth - 1, alpha, beta, false)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break # Beta cut-off
        return maxEval
    else:
        minEval = ∞
        for each child of node:
            eval = AlphaBeta(child, depth - 1, alpha, beta, true)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break # Alpha cut-off
        return minEval
```

In this pseudocode:

- The function **AlphaBeta** is called recursively.
- If the current node is a leaf node or if the depth limit is reached, the **evaluate** function is used to calculate the value of the node.
- If the current player is the **maximizing player**, the algorithm maximizes the evaluation value, and if the current player is the **minimizing player**, the algorithm minimizes the evaluation value.
- If **Beta ≤ Alpha**, pruning occurs and the search in the current branch is halted.

4. Optimality of Alpha-Beta Pruning

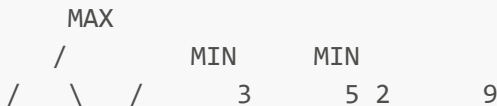
- **Optimality:** Alpha-Beta Pruning maintains the optimality of the **Minimax Algorithm**. That means the algorithm will still return the optimal move for the **MAX player**, just as Minimax would.
- **Efficiency:** The main benefit of Alpha-Beta Pruning is its efficiency. It can significantly reduce the number of nodes that need to be evaluated in the search tree. In the best case, Alpha-Beta Pruning can eliminate half of the nodes in the game tree, making the algorithm run much faster.

Time Complexity:

- Without Alpha-Beta Pruning, Minimax has a time complexity of **$O(b^d)$** , where **b** is the branching factor and **d** is the depth of the tree.
- With Alpha-Beta Pruning, the time complexity can be reduced to **$O(b^{(d/2)})$** in the best case, which makes it significantly faster, especially for large trees.

5. Example of Alpha-Beta Pruning in Action

Consider a game tree where the root node represents the current state of the game, and each edge represents a possible move for the players. Let’s assume the MAX player is trying to maximize the score, and the MIN player is trying to minimize the score.



- Initially, **Alpha = $-\infty$** and **Beta = ∞** .
- In the left subtree (MAX’s turn), the first MIN node has children 3 and 5. The minimum value is **3** (the left child).
- **Alpha is updated to 3**, as this is the best value for MAX found so far.
- In the right subtree (MIN’s turn), the first MIN node has children 2 and 9. The minimum value is **2**.
- Since **Alpha = 3** and **Beta = 2**, we know that the MAX player would not choose the 9 node, as it is guaranteed to be worse than the previously found branch.
- We **prune** the right subtree, saving time and resources.

This example demonstrates how Alpha-Beta Pruning eliminates the evaluation of certain branches and makes the search more efficient.

6. Classroom Discussion and Problem Solving

- **Open Discussion:** How does Alpha-Beta Pruning improve the performance of the Minimax algorithm in real-world games like Chess? What are some of the limitations of Alpha-Beta Pruning?
 - **Hands-on Activity:** Implement **Alpha-Beta Pruning** for a simple game (like Tic-Tac-Toe) and compare the performance with and without pruning. Analyze how much faster the algorithm runs when pruning is applied.
-

Conclusion:

- **Summary:** In this lecture, we explored **Alpha-Beta Pruning**, an optimization technique that improves the efficiency of the **Minimax Algorithm** by pruning branches of the game tree that cannot influence the final decision. We discussed how Alpha-Beta Pruning works, its optimality, and its significant impact on reducing the number of nodes evaluated in the search process.
- **Next Steps:** In the next lecture, we will explore more advanced optimization techniques and their applications in game-playing AI, such as **Monte Carlo Tree Search (MCTS)** and its use in complex game environments.